HW3 All-Pairs Shortest Path

114062640 謝秉宸

1.Implementation

1. Which algorithm do you choose in hw3-1?

與HW3-2、HW3-3相同，使用blocked Floyd-Warshall algorithm，並使用openMP來將資料分給不同thread。

2. How do you divide your data in hw3-2, hw3-3?

hw2

首先我先用 `(n / 64 + 1) * 64` 找出最小的大於n的64的倍數(我挑選64作為我的blocking factor)，然後將此數作為新的n，接著每個GPU的block處理64*64的數字

hw3

我採用列分解策略將工作負載分配給兩張 GPU。其中GPU 0 負責矩陣的上半部列、GPU 1 負責矩陣的下半部列，另外為了避免複雜的索引轉換，每張 GPU 都配置了N*N的空間，但實際上只計算並更新其負責的列。每回合更新後的 Pivot Row 會在 GPU 之間進行同步。

3.What's your configuration in hw3-2, hw3-3? And why? (e.g. blocking factor, #blocks, #threads)

Blocking Factor
設定值:64
因為每個int需要4byte的空間儲存，64*64*4byte=16KB這剛好能放入現代 GPU 的 Shared Memory（通常為 48KB 或更多），且還有空間儲存其他資料。並且因為64*64/1024=4，所以在phase1、2、3每個thread都要處理4或16個data
Thread Configuration (執行緒配置)
不同的 Phase 採用了不同的策略
Phase 1 & Phase 2:
- Grid：Phase 1 為 `(1, 1)`，Phase 2 為 `(Round, 2)`。
- Block Dim：`(32, 32)` (共 1024 threads)。

每個 Thread 處理 4 個元素
- 原因：

Phase 1 和 Phase 2 存在資料相依性（Data Dependency），需要頻繁的同步。使用標準的 Shared Memory 存取與較多的 Thread 數可以確保邏輯正確性並簡化實作。
Phase 3:
- Grid：`(Round, Round)`。
- Block Dim：`(16, 16)` (共 256 threads)。

- Work per Thread：4*4 矩陣 (16 個元素)。
- 原因：

Phase 3 是計算量最大的部分（佔總時間 90% 以上）。我使用了 2D Register Blocking每個 Thread 在 Register 中維護一個4*4的 `reg_results` 陣列。減少了 Shared Memory 的讀取次數。在最內層迴圈中，原本需要讀取 16 次 Shared Memory，透過 Register Blocking 只需要讀取 4+4=8個數值即可完成4*4=16次更新。

4. How do you implement the communication in hw3-3?

使用openMP分出兩條thread，接著用cudaDeviceEnablePeerAccess來讓兩個GPU可以互傳資料

```
int round = n / B;
    size_t size = (size_t)n * n * sizeof(int);

    #pragma omp parallel num_threads(2)
    {
        int gpu_id = omp_get_thread_num();
        cudaSetDevice(gpu_id);

        // 開啟P2P
        if (gpu_id == 0) cudaDeviceEnablePeerAccess(1, 0);
        else cudaDeviceEnablePeerAccess(0, 0);

        // 配置記憶體
        cudaMalloc(&Dist_GPU[gpu_id], size);
        cudaMemcpy(Dist_GPU[gpu_id], Dist, size, cudaMemcpyHostToDevice);

        int num_gpus = 2;
        int blocks_per_gpu = round / num_gpus;
        int start_round = gpu_id * blocks_per_gpu;
        int end_round = (gpu_id + 1) * blocks_per_gpu;
        if (gpu_id == num_gpus - 1) end_round = round;
        int my_height_blocks = end_round - start_round;
```

使用cudaMemcpyPeer傳送資料給另一個GPU

```
#pragma omp barrier
            if (gpu_id==owner_id) {
                size_t offset = (size_t)r * B * n * sizeof(int);
                size_t copy_size = (size_t)B * n * sizeof(int);
```

```
        cudaMemcpyPeer(
            (char*)Dist_GPU[1 - gpu_id] + offset, // Dest Addr (char* for byte offset)
            1 - gpu_id,              // Dest Device
            (char*)Dist_GPU[gpu_id] + offset,     // Src Addr
            gpu_id,               // Src Device
            copy_size                    // Bytes
        );
    }

    #pragma omp barrier
```

5. Briefly describe your implementations in diagrams, figures or sentences.

HW3-1

使用openMP，在cal function中用openMP做加速，並使用dynamic schedule(經實驗確認使用dynamic比static加速更多)

HW3-2

Pinned Memory (鎖定記憶體)：
使用 `cudaMallocHost` 分配 Host 記憶體。這允許 DMA 控制器直接在 Host 和 Device 之間傳輸數據，大幅提升了 `cudaMemcpy` 的頻寬。

2D Register Blocking:
Threads 協作（Coalesced Access）將 Pivot Column 和 Pivot Row 的資料從 Global Memory 載入到 Shared Memory (`s_i_k` , `s_k_j`)。

記憶體存取：

在 `FW_Phase1_2` 中，使用了 `unroll` 指令展開迴圈。並利用 `BlockIdx.y` 來判斷是 Row Block 還是 Col Block，避免編寫兩個分開的 Kernel

HW3-3

基本上實作與HW3-2大致相同，但一個GPU只需使用一半的data，多了將pivot row用cudaMemcpy傳到另一個GPU的步驟。

2. Profiling Results (hw3-2)

```
==2761749== Metric result:
Invocations                                Metric Name                      Metric Description         Min         Max         Avg
Device "NVIDIA GeForce GTX 1080 (0)"
    Kernel: FW_Phase1_2(int, int, int*, int)
        158                         achieved_occupancy                      Achieved Occupancy    0.497956    0.989081    0.739789
        158                               sm_efficiency                 Multiprocessor Activity       4.41%      96.77%      50.23%
        158                      shared_load_throughput            Shared Memory Load Throughput  118.04GB/s  2857.8GB/s  2416.8GB/s
        158                     shared_store_throughput           Shared Memory Store Throughput  40.366GB/s  977.28GB/s  826.46GB/s
        158                               gld_throughput                   Global Load Throughput  1.2232GB/s  29.615GB/s  25.044GB/s
        158                               gst_throughput                  Global Store Throughput  626.28MB/s  14.807GB/s  12.522GB/s
    Kernel: FW_Phase3(int, int, int*)
         79                         achieved_occupancy                      Achieved Occupancy    0.347702    0.349899    0.348666
         79                               sm_efficiency                 Multiprocessor Activity      99.27%      99.63%      99.49%
         79                      shared_load_throughput            Shared Memory Load Throughput  1735.5GB/s  1766.3GB/s  1751.7GB/s
         79                     shared_store_throughput           Shared Memory Store Throughput  144.62GB/s  147.19GB/s  145.97GB/s
         79                               gld_throughput                   Global Load Throughput  433.87GB/s  441.58GB/s  437.92GB/s
         79                               gst_throughput                  Global Store Throughput  289.25GB/s  294.39GB/s  291.95GB/s
```

使用c21.1作為測試資料，可以見到phase3的throughput、sm_efficiency皆比phase1、2大上許多，尤其是global throughput的部分，可見在phase3的優化還不錯

3.Experiment & Analysis (Nvidia GPU)

a. System Spec

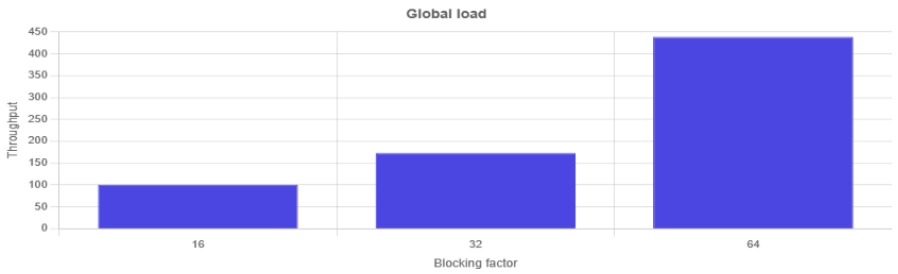
我使用課堂提供的GPU與CPU

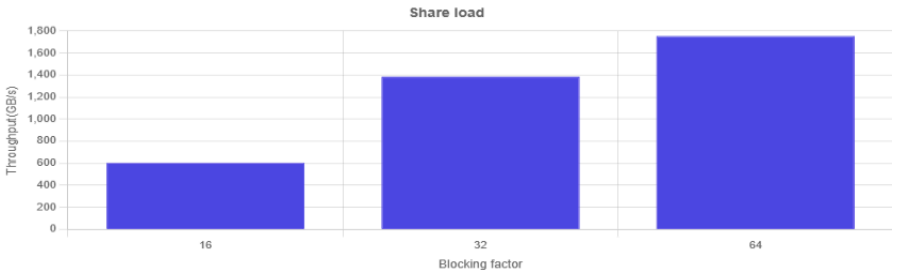
b. Blocking Factor (hw3-2)

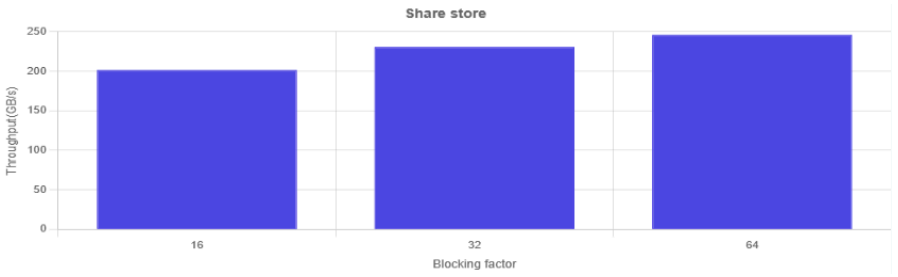
我用nvprof分別記錄B=16、B=32、B=64時phase3的數據並整理成如下圖表。

```
//64
==2761749== Metric result:
Invocations                                Metric Name                      Metric Description         Mi
n         Max         Avg
Device "NVIDIA GeForce GTX 1080 (0)"
    Kernel: FW_Phase3(int, int, int*)
         79                         achieved_occupancy                      Achieved Occupancy    0.3
47702    0.349899    0.348666
         79                               sm_efficiency                 Multiprocessor Activity      99.2
7%      99.63%      99.49%
         79                      shared_load_throughput            Shared Memory Load Throughp
ut  1735.5GB/s  1766.3GB/s  1751.7GB/s
         79                     shared_store_throughput           Shared Memory Store Throughp
ut  144.62GB/s  147.19GB/s  145.97GB/s
         79                               gld_throughput                   Global Load Throughput  433.87
GB/s  441.58GB/s  437.92GB/s
         79                               gst_throughput                  Global Store Throughput  289.25
GB/s  294.39GB/s  291.95GB/s

//32
==745265== Metric result:
Invocations                                Metric Name                      Metric Description         Mi
```

```
n        Max        Avg
Device "NVIDIA GeForce GTX 1080 (0)"
   Kernel: FW_Phase3(int, int, int*)
      157                    achieved_occupancy                    Achieved Occupancy    0.1
37037    0.137128    0.137061
      157                         sm_efficiency                Multiprocessor Activity      96.3
2%      96.37%      96.35%
      157                  shared_load_throughput            Shared Memory Load Throughp
ut  1335.8GB/s  1390.8GB/s  1384.3GB/s
      157                 shared_store_throughput           Shared Memory Store Throughp
ut  222.63GB/s  231.80GB/s  230.72GB/s
      157                        gld_throughput                 Global Load Throughput  166.97
GB/s  173.85GB/s  173.04GB/s
      157                        gst_throughput                Global Store Throughput  111.31G
B/s  115.90GB/s  115.36GB/s

//16
==1991321== Metric result:
Invocations                         Metric Name                     Metric Description         Mi
n        Max        Avg
Device "NVIDIA GeForce GTX 1080 (0)"
   Kernel: FW_Phase3(int, int, int*)
      249                    achieved_occupancy                    Achieved Occupancy    0.
090226    0.090358    0.090331
      249                         sm_efficiency                Multiprocessor Activity      93.7
1%      93.88%      93.86%
      249                  shared_load_throughput            Shared Memory Load Throughp
ut  602.98GB/s  607.17GB/s  604.49GB/s
      249                 shared_store_throughput           Shared Memory Store Throughp
ut  200.99GB/s  202.39GB/s  201.50GB/s
      249                        gld_throughput                 Global Load Throughput  100.5
0GB/s  101.20GB/s  100.75GB/s
      249                        gst_throughput                Global Store Throughput  50.24
9GB/s  50.598GB/s  50.374GB/s
```

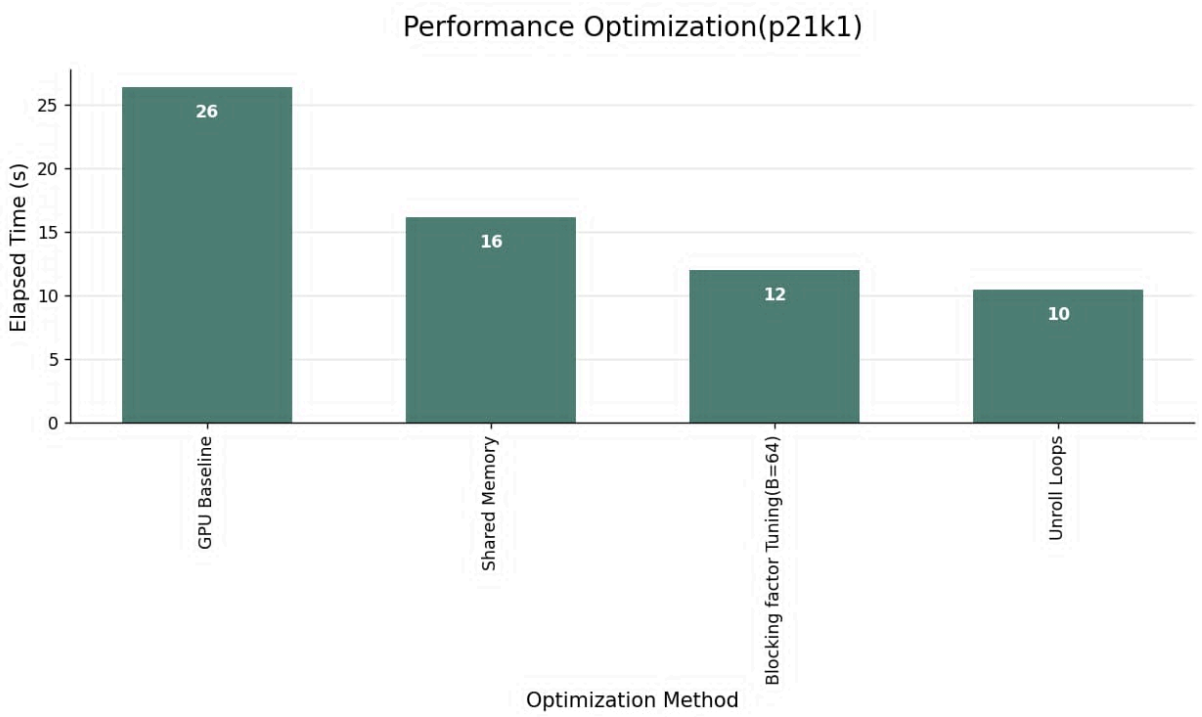






c. Optimization

使用p21k1當作測試資料，因為其花費時間較長，較難看出明顯差異



由圖可見最顯著的效能提升來自於將資料搬移至 Shared Memory。因為大幅減少了對高延遲 Global Memory 的存取次數，並利用 Shared Memory 的高頻寬特性來重複利用資料，解決了記憶體頻寬瓶頸。因此速度大幅提升。調整Blocking Factor 並發現 B=64 表現最佳。由於 GPU 的 Warp size 是 32，因此每條thread 要處理四筆資料從而提高了 Occupancy。最後，透過迴圈展開，我們消除了迴圈控制指令（如變數遞增、條件判斷）的開銷。這也讓編譯器能更有效地進行指令排程與暫存器分配，進一步加速。

d. Time Distribution (hw3-2)

使用 `<chrono>` 來記錄時間

測試資料為p11k1, p21k1與p28k1，三筆資料的N分別為11000、20959、28000

```
N, IO, Memcpy, Compute
11000, 1.903645, 0.076909, 1.120868
```

```
N, IO, Memcpy, Compute
20959, 4.646357, 0.278431, 7.553884
```

```
N, IO, Memcpy, Compute
28000, 7.473457, 0.495024, 17.883359
```

在N=11000時，IO time還大於compute time，但可以注意到N=20959時，compute time已是佔比最大的部分了。由上面三筆數據可以發現，隨著N越大，compute的佔比也會隨之上升，因為Floyd-Warshall是O(N^3)的演算法，所以compute time的增長才會如此快速

而若想再更進一步將秒數壓低，我想應該要減少的是IO time ，然而我暫時還沒有想出更佳的解法

5. Experience & conclusion

在用 GPU 加速的 Floyd-Warshall 演算法時，最大的挑戰在於有效管理記憶體階層 。一開始將 blocked Floyd-Warshall對應到 CUDA thread 索引上相當容易出錯，特別是在處理 Shared Memory 的存取座標的時候。在實作運算量最大的 Phase 3 時，我深刻體會到如果只依賴 Global Memory 會造成巨大的效能瓶頸。
另一個重要的學習點是Blocking Factor 的調整。雖然可以想到B=64的時候可能是最快的，但也真的要一個一個去試才能確認。